

Equivalence with Disjunction Elimination and Extension Types

Everett (Trey) Plante III

May 5, 2025

What is this about?

- Providing the basis for an implementation of a *logically-indexed* dependent type theory.
- Introducing λ^Φ and building an algorithm for deciding the equivalence of terms in λ^Φ .

The λ -calculus is a notational system introduced by Alonzo Church for describing functions and substitution (capture avoiding):

$$(\lambda x.M) N \equiv M(N/x)$$

The λ -calculus is the basis for a family of programming languages called functional programming languages. Substitution is computation:

$$(\lambda x.x + 5) 5 \equiv (x + 5) (5/x) \equiv 5 + 5 \equiv 10$$

Simply Typed λ -calculus

In the λ -calculus, erroneous function applications can be written: $(\lambda x. x + 5)$ **False**. This creates errors in programs that aren't realized until a program is run. Types enable computers to catch errors before running a program:

$$(\lambda x : \mathbb{N}. x + 5) \text{ False}$$

The notion of *type theory* colloquially refers to variants of the λ -calculus with types.

Type Theory for Mechanized Mathematics

A type theory has two principal statements called judgments:

$$\Gamma \vdash M : \tau \text{ and } \Gamma \vdash M \equiv N : \tau$$

In type theory, there are rules to conclude judgments from other hypothesized judgments:

$$\frac{\Gamma, x : \tau_1 \vdash M : \tau_2}{\Gamma \vdash (\lambda x : \tau_1. M) : \tau_1 \rightarrow \tau_2}$$

One can build derivations out of these rules, and these derivations make up proofs. This enables computers to check proofs by checking type-theoretic derivations. Derivations in the simply typed λ -calculus actually correspond to statements in propositional logic (propositions-as-types).

Proof Assistants

The synergy between functional programming and formalized mathematics has given rise to proof assistants. These systems give mathematicians interactive blackboards and computer scientists trust in their software.

```
theorem non_tail_sum_eq_tail_sum : NonTail.sum = Tail.sum
  :=
  by funext xs
  simp [Tail.sum]
  rw [←Nat.zero_add (NonTail.sum xs)]
  exact non_tail_sum_eq_helper_accum xs 0
```

A user writes statements by composing types and proves them by providing a term of a type.

Dependent Types

Dependent types are a generalization of simple types, where types can depend on terms introduced by Martin-Loef. Type families:

$$\Gamma, x : \tau_1 \vdash \tau_2(x)$$

```
inductive Vect ( $\alpha$ : Type) : Nat  $\rightarrow$  Type where
| nil : Vect  $\alpha$  0
| cons:  $\alpha \rightarrow$  Vect  $\alpha$  n  $\rightarrow$  Vect  $\alpha$  (n + 1)

def one_nat : Vector Nat 1 := cons 1 nil
```

Dependent Functions

The first upgrade over the simply typed λ -calculus for λ^Φ is dependent functions.

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2(x) \quad \Gamma \vdash (x : \tau_1) \rightarrow \tau_2(x) : \mathcal{U}_l}{\Gamma \vdash \lambda x : \tau_1. e : (x : \tau_1) \rightarrow \tau_2(x)}$$

$$\frac{\Gamma, x : \tau_1 \vdash M : \tau_2(x) \quad \Gamma \vdash N : \tau_1}{\Gamma \vdash (\lambda x : \tau_1. M)N \equiv M(N/x) : \tau_2(N/x)}$$

Why dependent types?

Dependent types have a lot of expressive power.

```
theorem schroeder_bernstein {f :  $\alpha \rightarrow \beta$ } {g :  $\beta \rightarrow \alpha$ } (hf :  
  Injective f) (hg : Injective g) :  
   $\exists h : \alpha \rightarrow \beta$ , Bijective h :=  
  ⟨sbFun f g, sb_injective f g hf, sb_surjective f g hg⟩
```

Dependent Types for Programming

A programmer can express complex invariants with types.

```
def f (b : Bool) : if b then Int else Nat :=  
  match b with  
  | true => -1 : Int  
  | false => 0 : Nat
```

Dependent Types for Math

A mathematician can express propositions with types.

```
variable ( $\alpha$  : Type) (p q :  $\alpha \rightarrow \text{Prop}$ )
```

```
example : ( $\forall x, p\ x \wedge q\ x$ )  $\leftrightarrow$  ( $\forall x, p\ x$ )  $\wedge$  ( $\forall x, q\ x$ ) :=
```

```
Iff.intro
```

```
(fun h => ⟨fun x => (h x).left, fun x => (h x).right⟩)
```

```
(fun h x => ⟨h.1 x, h.2 x⟩)
```

The next upgrade for λ^Φ is the addition of logical types. The logical types consist of a term type $\mathcal{T}$ and a formula type $\mathcal{F}$, corresponding to the terms and formulas of some logical language.

$$\begin{array}{c}
 \overline{\vdash c : \mathcal{T}} \quad \overline{\vdash x_i : \mathcal{T}} \\
 \\
 \overline{\vdash \perp : \mathcal{F}} \quad \overline{\vdash \top : \mathcal{F}} \quad \frac{\Psi \mid \Theta \vdash t_1 : \mathcal{T} \quad \dots \quad \Psi \mid \Theta \vdash t_n : \mathcal{T}}{\Psi \mid \Theta \vdash R(t_1, \dots, t_n) : \mathcal{F}} \\
 \\
 \frac{\Psi \mid \Theta \vdash \alpha : \mathcal{F} \quad \Psi \mid \Theta \vdash \beta : \mathcal{F}}{\Psi \mid \Theta \vdash \alpha \wedge \beta : \mathcal{F}} \quad \frac{\Psi \mid \Theta \vdash \alpha : \mathcal{F} \quad \Psi \mid \Theta \vdash \beta : \mathcal{F}}{\Psi \mid \Theta \vdash \alpha \vee \beta : \mathcal{F}}
 \end{array}$$

These are not first-class types.

Entailment Judgment

The logic layer interacts with terms and types of λ^Φ through an entailment judgment.

$$\Gamma \vdash \alpha$$

Semantic Requirements

The semantics imposed on the logic keep the class of logics very general.

- $\alpha \vee \beta \vdash \psi$ iff $\alpha \vdash \psi$ and $\beta \vdash \psi$.
- $\phi \vdash \alpha \wedge \beta$ iff $\phi \vdash \alpha$ and $\phi \vdash \beta$.
- For all substitutions σ of the variables and parameters in context, $\phi \vdash \psi$ iff $\phi\sigma \vdash \psi\sigma$.
- If $\phi \vdash \alpha$ and $\alpha \vdash \psi$, then $\phi \vdash \psi$.
- $\alpha \vdash \alpha$
- $\alpha \wedge \beta \vdash \alpha$ and $\alpha \wedge \beta \vdash \beta$

Propositional Logic

One example of a logic that fits the requirements of λ^Φ is propositional logic.

$$\overline{\vdash 0, 1 : \mathcal{T}} \quad \overline{\vdash x_i : \mathcal{T}}$$

$$\frac{\Psi \mid \Theta \vdash t : \mathcal{T}}{\Psi \mid \Theta \vdash t : \mathcal{F}}$$

$$\Psi \mid \Theta \vdash t : \mathcal{F}$$

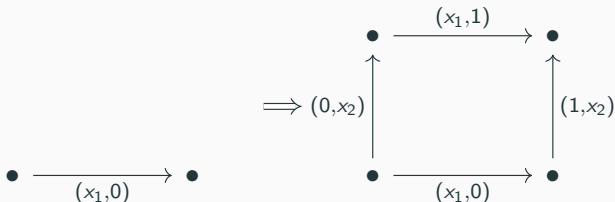
$$\frac{\Psi \mid \Theta \vdash \alpha : \mathcal{F} \quad \Psi \mid \Theta \vdash \beta : \mathcal{F}}{\Psi \mid \Theta \vdash \alpha \wedge \beta : \mathcal{F}}$$

$$\frac{\Psi \mid \Theta \vdash \alpha : \mathcal{F} \quad \Psi \mid \Theta \vdash \beta : \mathcal{F}}{\Psi \mid \Theta \vdash \alpha \vee \beta : \mathcal{F}}$$

$$\frac{\Psi \mid \Theta \vdash \alpha : \mathcal{F}}{\Psi \mid \Theta \vdash \neg \alpha : \mathcal{F}}$$

Logics for Cubical Type Theory

The motivation behind this work is the cofibration logics of cubical type theories. In this setting, the cofibration logic is an intuitionistic logic in which n -dimensional subcubes are constructed.



Identity types

The introduction of logics into type theory enables the construction of an identity (path) type:

$$\Gamma \vdash p : Id(M, N, \tau)$$

$$\Gamma \vdash p@0 \equiv M : \tau$$

$$\Gamma \vdash p@1 \equiv N : \tau$$

Higher Inductive Types I

Identity types allow programmers to model invariants at datatype definitions.

```
inductive Int where
```

```
| Pos : Nat -> Int
```

```
| Neg : Nat -> Int
```

```
higher inductive Int' where
```

```
| Pos : Nat -> Int
```

```
| Neg : Nat -> Int
```

```
| Invariant : Id(Pos zero, Neg zero, Int)
```

Higher Inductive Types II

Mathematicians can model complex objects like the circle using HITs.

```
higher inductive S1 where
| base : S1
| loop : Id(base, base, S1)
```

People have used this to prove cool theorems in homotopy theory.

Extension Type

$$\frac{\Gamma \vdash \tau : \mathcal{U}_I \quad \Gamma \vdash \psi : \mathcal{F} \quad \Gamma \vdash t : [\psi] \rightarrow \tau}{\Gamma \vdash \tau[\psi \rightarrow t] : \mathcal{U}_I} \quad \frac{\Gamma \vdash a : \tau[\psi \rightarrow t]}{\Gamma \vdash \text{out}_{\psi}^t(a) : \tau}$$

$$\frac{\Gamma \vdash a : \tau \quad \Gamma \vdash \psi : \mathcal{F} \quad \Gamma, \psi \vdash t : \tau \quad \Gamma, \psi \vdash t \equiv a : \tau \quad \Gamma \vdash \tau : \mathcal{U}_I}{\Gamma \vdash \text{in}_{\psi}^t(a) : \tau[\psi \rightarrow t]}$$

$$\frac{\Gamma \vdash \text{in}_{\psi}^t(a) : \tau}{\Gamma \vdash \text{out}_{\psi}^t(\text{in}_{\psi}^t(a)) \equiv a : \tau} \quad \frac{\Gamma \vdash a : \tau[\psi \rightarrow t] \quad \Gamma \vdash \psi}{\Gamma \vdash \text{out}_{\psi}^t(a) \equiv t : \tau}$$

Riehl and Shulman. A type theory for synthetic ∞ -categories.

Disjunction Elimination

$$\frac{\Gamma \vdash \alpha \vee \beta \quad \Gamma, \alpha \vdash e_1 : \tau \quad \Gamma, \beta \vdash e_2 : \tau \quad \Gamma, \alpha, \beta \vdash e_1 \equiv e_2 : \tau}{\Gamma \vdash [e_1, e_2]_{\alpha, \beta} : \tau}$$

$$\frac{\Gamma \vdash \alpha_i \quad \Gamma, \alpha_1 \vdash e_1 : \tau \quad \Gamma, \alpha_2 \vdash e_2 : \tau \quad \Gamma, \alpha_1, \alpha_2 \vdash e_1 \equiv e_2 : \tau}{\Gamma \vdash [e_1, e_2]_{\alpha_1, \alpha_2} \equiv e_i : \tau}$$

$$\frac{\Gamma \vdash \alpha \vee \beta \quad \Gamma \vdash e : \tau}{\Gamma \vdash e \equiv [e, e]_{\alpha, \beta} : \tau}$$

$$\frac{\Gamma \vdash \alpha \leftrightarrow \gamma \quad \Gamma \vdash \beta \leftrightarrow \delta \quad \Gamma, \alpha \vdash e_1 \equiv f_1 : \tau \quad \Gamma, \beta \vdash e_2 \equiv f_2 : \tau}{\Gamma \vdash [e_1, e_2]_{\alpha, \beta} \equiv [f_1, f_2]_{\gamma, \delta} : \tau}$$

Motivation

- λ^Φ is the core calculus for cubical and simplicial type theories.
- Cubical and simplicial type theories enable the formalization of cutting edge math.
- The original motivation is an implementation of a directed cubical type theory [WL20].

- We have a dependent type theory with some logical formulas and extension types and disjunction eliminations.
- We want to decide the equational theory of this new type system i.e. give a procedure for deciding the judgment $\Gamma \vdash M \equiv N : \tau$.
- Why?

An implementation of type theory largely amounts to an algorithm that can check the types of terms. This means building an algorithm that decides the judgment $M : \tau$. This requires the ability to decide the definitional equality judgment for types because of the type conversion rule:

$$\frac{\Gamma \vdash M : \tau \quad \Gamma \vdash \tau \equiv \tau' : \mathcal{U}_I}{\Gamma \vdash M : \tau'}$$

Because types can depend on terms, it is necessary to decide the equivalence of arbitrary terms: $\Gamma \vdash M \equiv N : \tau$.

Deciding Equivalence

Deciding equivalence is also important to reduce the proof obligations on the user of a proof assistant.

$$\mathbf{isOdd}(1 + 2 + 2) \equiv \mathbf{isOdd}(5)$$

The equational theory of any type theory must be specified in order to capture as many equivalences as possible, and ensure decidability. If definitional equality is undecidable, so is type-checking.

How could we decide equivalence?

- Normalize terms such that two definitionally equal terms are sent to identical normal forms. This could be impractical and is difficult given disjunction eliminations.
- Design a binary equivalence algorithm that compares terms and stops if the terms are deemed different. This is the approach we take.

Next steps

- Develop an algorithm that provides the infrastructure for type-checking λ^Φ . This will be an equivalence algorithm that decides the equational theory of λ^Φ .
- Show that the algorithm is correct:
 1. If $\Gamma \vdash M \equiv N : \tau$ then $\Gamma \triangleright M \Leftrightarrow N : \tau$.
 2. If $\Gamma \vdash M : \tau$, $\Gamma \vdash N : \tau$, and $\Gamma \triangleright M \Leftrightarrow N : \tau$, then $\Gamma \vdash M \equiv N : \tau$.

Equational theory of λ^Φ : Disjunction Extensionality

$$\frac{\Gamma \vdash \alpha \vee \beta \quad \Gamma, \alpha \vdash M \equiv N : \tau \quad \Gamma, \beta \vdash M \equiv N : \tau}{\Gamma \vdash M \equiv N : \tau}$$

First Attempt: Reduce-and-Compare I

We will define a notion of reduction, along with its reflexive-transitive closure, $M \Downarrow$.

$$\begin{array}{ll} \Phi \triangleright (\lambda x. e) f \rightarrow e(f/x) & \\ \Phi \triangleright ef \rightarrow e'f & \text{if } \Phi \triangleright e \rightarrow e' \\ \Phi \triangleright [e, f]_{\alpha, \beta} \rightarrow e & \text{if } \Phi \vdash \alpha \\ \Phi \triangleright [e, f]_{\alpha, \beta} \rightarrow f & \text{if } \Phi \vdash \beta \\ \Phi \triangleright \text{out}_{\psi}^t(a) \rightarrow t & \text{if } \Phi \vdash \psi \\ \Phi \triangleright \text{out}_{\psi}^t(\text{in}_{\psi}^t(a)) \rightarrow a & \\ \Phi \triangleright \text{out}_{\psi}^t(a) \rightarrow a' & \text{if } \Phi \triangleright a \rightarrow a' \end{array}$$

First Attempt: Reduce-and-Compare II

We define an algorithmic equivalence procedure $\Phi \triangleright M \Leftrightarrow N$.

$\Phi \triangleright M \Leftrightarrow N$ iff

- $\Phi \triangleright M \Downarrow = N \Downarrow$.

Counterexamples:

1. $\Phi \triangleright (\lambda x : \tau. fx) \not\Leftrightarrow f$
2. $\alpha \vee \beta \triangleright [M, N]_{\alpha, \beta} \not\Leftrightarrow [N, M]_{\beta, \alpha}$
3. $\alpha \vee \beta \triangleright M \not\Leftrightarrow [M, M]_{\alpha, \beta}$

Second Attempt: General Strategy

- Algorithmic Equivalence is split into two mutually defined functions: type-directed equivalence, $\Gamma \triangleright M \Leftrightarrow N : \tau$, and structural equivalence, $\Gamma \triangleright p \Leftrightarrow q \uparrow \tau$.
- Type-directed equivalence applies eliminators until the type reaches a “base” type.
- Structural equivalence structurally compares fully-reduced terms and calls type-directed equivalence on non-reduced terms.

Second Attempt: Type-Directed Equivalence

$\Gamma \triangleright M \Leftrightarrow N : \tau$ if τ is base and $\Gamma \triangleright M \Downarrow \Leftrightarrow N \Downarrow$

$\Gamma \triangleright M \Leftrightarrow N : \tau[\psi \rightarrow t]$ if $\Gamma \triangleright \text{out}_{\psi}^t(M) \Leftrightarrow \text{out}_{\psi}^t(N) : \tau$

$\Gamma \triangleright M \Leftrightarrow N : (x : \tau_1) \rightarrow \tau_2(x)$ if $\Gamma, x : \tau_1 \triangleright Mx \Leftrightarrow Nx : \tau_2(x)$

Second Attempt: Structural Equivalence

$$\Gamma \triangleright x \leftrightarrow x \uparrow \Gamma(x)$$

$$\Gamma \triangleright pM \leftrightarrow qN \uparrow \tau_2(M/x) \quad \text{if } \Gamma \triangleright p \leftrightarrow q \uparrow (x : \tau_1) \rightarrow \tau_2(x),$$

$$\Gamma \triangleright M \leftrightarrow N : \tau_1$$

$$\Gamma \triangleright \text{out}_{\psi}^t(p) \leftrightarrow \text{out}_{\psi'}^{t'}(q) \uparrow \tau \quad \text{if } \Gamma \triangleright p \leftrightarrow q \uparrow \tau[\psi \rightarrow t],$$

$$\Gamma \triangleright t \leftrightarrow t' : \tau,$$

$$\Gamma \triangleright \psi \leftrightarrow \psi'$$

Back to the counter examples

We've made some progress.

- $\Gamma \triangleright (\lambda x : \tau_1. fx) \Leftrightarrow f : (x : \tau_1) \rightarrow \tau_2(x)$ if
 $\Gamma, x : \tau_1 \triangleright (\lambda x : \tau_1. fx)x \Leftrightarrow fx : \tau_2(x)$
- $\Gamma \triangleright [M, N]_{\alpha, \beta} \not\Leftrightarrow [N, M]_{\beta, \alpha}$
- $\Gamma \triangleright M \not\Leftrightarrow [M, M]_{\alpha, \beta}$

Third Attempt: Formula Guarded Reduction

Formula guarded reduction attaches expressions to a precondition that forces that reduction.

$$(\lambda x : \tau. e) f \Rightarrow (e(f/x), \top)$$

$$ef \Rightarrow (e_1 f, \epsilon_1), \dots, (e_n f, \epsilon_n) \quad \text{if } e \Rightarrow \vec{e}$$

$$[e, f]_{\alpha, \beta} \Rightarrow (e, \alpha), (f, \beta)$$

$$\text{out}_{\psi}^t(a) \Rightarrow (t, \psi), (\text{out}_{\psi}^t(a_1), \alpha_1), \dots, (\text{out}_{\psi}^t(a_n), \alpha_n) \quad \text{if } a \Rightarrow \vec{a}$$

$$\text{out}_{\psi}^t(a) \Rightarrow (t, \psi)$$

$$\text{out}_{\psi}^t(\text{in}_{\psi}^t(a)) \Rightarrow (a, \top)$$

Disjunction Extensionality at Base Types

Formula guarded reduction is used to mimic the disjunction extensionality rule at base types.

$$\frac{M \Rightarrow \vec{M} \quad N \Rightarrow \vec{N} \quad \Gamma \vdash \bigvee_{i,j} \mu_i \wedge \eta_j \quad \forall i,j. \Gamma, \mu_i, \eta_i \triangleright M_i \Leftrightarrow N_j : \tau}{\Gamma \triangleright M \Leftrightarrow N : \tau}$$

Back to the counter examples I

$$\frac{[M, M]_{\alpha, \beta} \Rightarrow (M, \alpha), (M, \beta) \quad \Gamma \alpha \triangleright M \Leftrightarrow M : \tau \quad \Gamma, \beta \triangleright M \Leftrightarrow M : \tau}{\Gamma \triangleright M \Leftrightarrow [M, M]_{\alpha, \beta} : \tau}$$

Back to the counter examples II

$$\begin{array}{c}
 \begin{array}{c}
 [M, N]_{\alpha, \beta} \Rightarrow (M, \alpha), (N, \alpha) \\
 [N, M]_{\beta, \alpha} \Rightarrow (N, \beta), (M, \alpha)
 \end{array}
 \quad
 \frac{
 \begin{array}{c}
 \Gamma, \alpha, \beta \triangleright M \Leftrightarrow N : \tau \\
 \Gamma, \alpha, \alpha \triangleright M \Leftrightarrow M : \tau
 \end{array}
 }{
 \Gamma, \alpha \triangleright M \Leftrightarrow [N, M]_{\beta, \alpha} : \tau
 }
 \quad
 \frac{
 \begin{array}{c}
 \Gamma, \beta, \beta \triangleright N \Leftrightarrow N : \tau \\
 \Gamma, \beta, \alpha \triangleright N \Leftrightarrow M : \tau
 \end{array}
 }{
 \Gamma, \beta \triangleright N \Leftrightarrow [N, M]_{\beta, \alpha} : \tau
 }
 \\
 \hline
 \Gamma \triangleright [M, N]_{\alpha, \beta} \Leftrightarrow [N, M]_{\beta, \alpha} : \tau
 \end{array}$$

Theorem (Disjunction Extensionality)

If $\forall i. \Gamma, \alpha_i \triangleright M \Leftrightarrow N : \tau$ and $\Gamma \vdash \bigvee_i \alpha_i$, then $\Gamma \triangleright M \Leftrightarrow N : \tau$.

Theorem (Soundness of Formula Guarded Reduction)

If $\Gamma \vdash M : \tau$ and $\Gamma \vdash M \Rightarrow \vec{M}$, then $\Gamma, \mu_i \vdash M \equiv M_i : \tau$ and $\Gamma, \mu_i \vdash M_i : \tau$.

Lemma (Transitivity)

In the following, let the terms submitted to algorithmic equality be well-typed. If $\Gamma \mid \delta \triangleright M \Leftrightarrow N : \tau$ and $\Gamma \mid \gamma \triangleright N \Leftrightarrow O : \tau$, then $\Gamma \mid \delta \wedge \gamma \triangleright M \Leftrightarrow O : \tau$.

Lemma (Symmetry)

In the following, let the terms submitted to algorithmic equality be well-typed. If $\Gamma \triangleright M \Leftrightarrow N : \tau$, then $\Gamma \triangleright N \Leftrightarrow M : \tau$.

Theorem (Soundness of Algorithmic Equality)

Let $\Gamma \vdash M : \tau$ and $\Gamma \vdash N : \tau$. If $\Gamma \triangleright M \Leftrightarrow N : \tau$, then $\Gamma \vdash M \equiv N : \tau$.

Theorem (Completeness of Algorithmic Equivalence)

If $\Gamma \vdash M \equiv N : \tau$, then $\Gamma \triangleright M \Leftrightarrow N : \tau$.

How??

Two Kripke logical relations; one for proving results about λ^Φ and one for establishing the connection between λ^Φ and algorithmic equivalence.

- $\Gamma \vdash J \implies \Gamma \Vdash_J^s J \implies \Gamma \vdash J$
- $\Gamma \vdash M \equiv N : \tau \implies \Gamma \Vdash_J^c M = N : \tau \implies \Gamma \triangleright M \Leftrightarrow N : \tau.$

- Design a type-checker. This involves designing a judgmental structure for checking and synthesizing types, and proving the judgmental adequately presents the judgments of λ^Φ .
- Normalization by Evaluation. This is another approach to deciding equivalence that evaluates terms into a semantic domain and then quotes them back into syntactic normal forms.
- Loosen the restrictions on the logical types. Can we type-check a type theory where $\mathcal{T}$ and $\mathcal{F}$ are first class types?

Thank you! Questions?

References

- [WL20] Matthew Z. Weaver and Daniel R. Licata, *A constructive model of directed univalence in bicubical sets*, Proceedings of the 35th annual ACM/IEEE symposium on logic in computer science, July 8, 2020, pp. 915–928.